Thamill Indian B U
Cohort-15 (AI Engineer)

Day 12 Engineering Reflection: Reliable Agentic Workflows

AI Engineering Program | Day 12: Moving Critical Rules out of Prompts into Deterministic Code
Repository: https://github.com/ThamillIndianBlend360/AI-Assignments/tree/main/Day12

_

1. Overview & Core Objective

The goal of Day 12 was to refactor the agentic workflow from Day 11 to achieve industrial reliability. Rather than relying on prompt instructions to enforce business rules, critical logic (such as leave balance checks, duplicate handling, input validation, and audit logging) was shifted entirely into deterministic Python backend code.

2. Verification & Execution Evidence

The test suite executed in Day12/main.py verified that every reliability guardrail operated as expected:

```
Created request: {'status': 'APPROVED', 'request_id': 'LR-1001', 'remaining_balance': 1}
PASS: Idempotency - duplicate request did not deduct leave again.
PASS: Reusing a key with changed arguments was rejected.
PASS: Balance enforcement - insufficient balance blocked the request.
PASS: Pydantic rejected invalid days, types, and dates.
Audit trail saved to: C:\projects\AI Engineering\Project1\day12_audit_trail.log
```

3. Key Reliability Techniques Implemented

1. Idempotency Key Tracking: Requests are keyed by user ID and date. Re-submitting identical requests returns the existing approval status instantly without re-deducting leave balances.

2. Deterministic Balance Enforcement: The LLM prompt is no longer trusted to compute numbers. The create_leave_request backend function evaluates user balances programmatically and rejects requests when balances are insufficient.

3. Strict Pydantic Argument Guardrails: Input dates, numeric ranges (e.g., positive leave days), and string parameter structures are strictly type-checked before any backend invocation.

4. Audit Trail Logging: All agent thoughts, tool decisions, execution results, and validation failures are logged to day12_audit_trail.log for complete enterprise traceability.

4. Engineering Reasoning & Trade-Offs

• **Architectural Decision:** Shift business rules from system prompts to deterministic software boundaries.
• **Alternative Considered:** Relying on detailed LLM system prompts and multi-shot examples.
• **Trade-off Accepted:** Writing extra backend boilerplate code in exchange for 100% deterministic safety, preventing double-booking and math errors.